

TalentAI

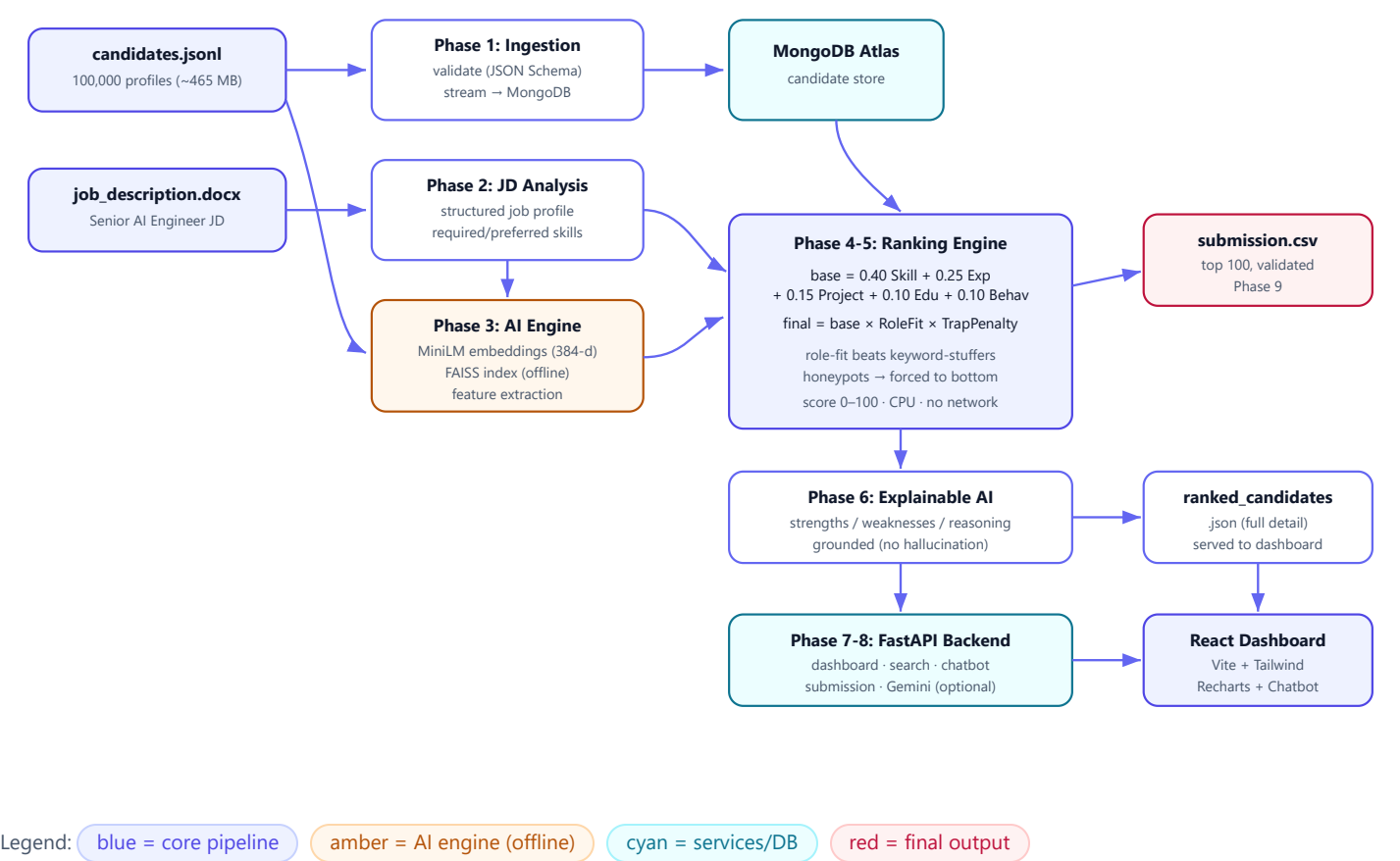
Intelligent Candidate Discovery & Ranking System | Redrob "India Runs" Data & AI Challenge

Project guide, architecture & interview preparation · Role ranked for: **Senior AI Engineer (Founding Team)**

60-second pitch: TalentAI is an explainable candidate ranking engine. The dataset is full of traps (keyword-stuffers, honeypots, perfect-on-paper-but-wrong-fit profiles), so instead of naive keyword or embedding matching, we score every candidate on **five weighted signals**, then multiply by a **role-fit** signal and a **trap-penalty** that encode what the JD *means*, not just what it *says*. It runs on **CPU, no network, in under a minute** over all 100,000 candidates, and every ranking ships with a grounded, human-readable explanation. **Zero honeypots reach our top 100.**

1. System Architecture

How every part connects, from raw data to the recruiter dashboard.



2. End-to-End Workflow (JD → ranked output)

- Ingest** — load `candidates.jsonl`, validate each record against `candidate_schema.json`, store in MongoDB (streamed, flat memory).
- JD Analysis** — parse the JD into a structured profile (required/preferred skills, disqualifiers).
- Candidate Understanding** — extract skills, experience, projects, education, behavioral signals; build embeddings (offline).
- Matching** — semantic similarity (cosine) + 5 component scores.
- Ranking** — apply role-fit & trap modifiers → ordered top 100 (sort by score desc, candidate_id asc).
- Explainability** — generate grounded reasoning per candidate.
- Output** — validated `submission.csv` + dashboard & AI chatbot.

3. The Ranking Formula (the heart of the project)

$$\begin{aligned} \text{base} &= 0.40 \times \text{Skill} + 0.25 \times \text{Experience} + 0.15 \times \text{Project} \\ &\quad + 0.10 \times \text{Education} + 0.10 \times \text{Behavioral} \end{aligned}$$
$$\text{final} = \text{base} \times (0.35 + 0.65 \times \text{RoleFit}) \times \text{TrapPenalty} \quad \rightarrow \text{scaled to } 0\text{--}100$$

Component	Weight	What it measures
Skill Match	40%	JD-concept coverage, gated by a "trust" factor (endorsements + months used) so padded skill lists are discounted; blended with semantic similarity.
Experience	25%	Curve peaking at the ideal 6–8 yrs (5–9 range) + bonus for product companies over IT-services.
Project Relevance	15%	Did they actually build ranking / search / recsys / retrieval (read from career descriptions).
Education	10%	Institution tier + relevant field (kept light — JD welcomes plain-language fits).
Behavioral	10%	Recruiter response rate, recency, open-to-work, interview completion, GitHub activity (availability).
RoleFit (×)	mult.	Decisive anti-keyword-stuffer signal — title + real career evidence.
TrapPenalty (×)	mult.	Honeypots → 0; soft penalties for consulting-only, job-hoppers, research-only, CV/speech-without-NLP, inactive.

Key line: RoleFit and TrapPenalty are *multipliers, not additions* — so even a perfect keyword score can't rescue a fraudulent or wrong-role profile.

4. The 3 Most Important Parts

1. Role-Fit + Trap Detection

We see beyond keywords and reject fake / wrong-role profiles. **Our biggest differentiator** vs traditional matching.

2. 5-Component Weighted Score

Combines all candidate signals (profile + career + behavioral) into one trustworthy 0–100 score.

3. Embeddings + Compute-Safe Design

Real semantic AI, but runs on CPU in under a minute — production-ready, not a benchmark toy.

Honeypot Safety

Challenge disqualifies if >10% honeypots in top 100. **Ours = 0.**

5. AI / ML Topics (what & where used)

Topic	What it is	Used for in this project
Embeddings	Text → numeric vector; similar meanings sit close together.	Semantic understanding of profiles & JD.
Cosine similarity	Angle between two vectors (1 = same meaning).	Compare candidate vector to JD vector.
Semantic vs keyword search	Meaning-based vs exact-word match.	Catches "built a recommender" even without the word "RAG".
all-MiniLM-L6-v2	Small, fast embedding model (384-d).	Chosen to fit CPU + 5-min budget.
FAISS	Fast vector similarity search library.	"Find similar candidates" / semantic search.
Hybrid retrieval	Dense (embeddings) + sparse (rules/keywords).	Each catches what the other misses.
NDCG / MRR / MAP / P@10	Ranking quality metrics.	How judges score us (NDCG@10 = 50%).
RAG / LLMs	Retrieve context then prompt an LLM.	Understood; not called per-candidate (won't scale in 5 min).
Honeypot detection	Spot "impossible" profiles.	Force fake profiles to the bottom.
Hallucination prevention	Only state facts from the profile.	Trustworthy reasoning for Stage-4 review.

6. Full Tech Stack (name + why)

Layer	Technology	Why chosen
Frontend	React, Vite, Tailwind CSS, Recharts, Axios	Fast modern recruiter dashboard with charts; hooks for state.
Backend	FastAPI, Uvicorn, Pydantic	Async Python APIs with auto-generated docs & typed validation.
AI / ML	Sentence-Transformers (MiniLM), FAISS, NumPy	CPU-fast embeddings + vector search; NumPy for cosine math.
Explanations	Rule-based + optional Gemini	Grounded, no hallucination; LLM used offline only.
Database	MongoDB Atlas + PyMongo	Flexible NoSQL for nested candidate documents.
Data / Validation	JSON Schema (jsonschema), pytest	Spec-compliant ingestion + 15 tests proving real engineering.
DevOps	Docker, docker-compose	Reproducible deployment for Stage-3.

7. Key Design Decisions (defend these)

- **Why not an LLM per candidate?** No network, CPU only, 5-min budget — 100K LLM calls can't scale; a small ranker over precomputed features can.
- **Why MiniLM over a large model?** Quality is good enough and it runs fast on CPU within budget.
- **Why rule-based + semantic hybrid?** Rules encode the JD's explicit intent (traps, role-fit); embeddings catch meaning. Together they beat either alone.
- **Why precompute embeddings offline?** The spec allows offline pre-computation; the ranking step then loads cached vectors and stays under 5 min.
- **Why multipliers for role-fit & traps?** They must be able to *override* a high keyword score, not just add to it.

8. Likely Judge Questions & Crisp Answers

Question	Answer
How do you go beyond keyword matching?	Role-fit (title + career evidence) + semantic embeddings + behavioral signals.
How do you handle honeypots?	Integrity checks (impossible facts) force them to the bottom — 0 in our top 100.
How do you prevent hallucinated reasoning?	Reasoning is built only from the candidate's real profile fields.
How do you meet compute constraints?	Embeddings precomputed offline; ranking streams + uses NumPy; ~36s for 100K, no network.
Why these weights?	Skill dominates as the core need, but role-fit/traps gate it multiplicatively.
What would you improve?	Learning-to-rank (XGBoost), full semantic hybrid at scale, broader honeypot recall.
How does it scale to 200K+?	Swap FAISS FlatIP for IVF/HNSW (approx. nearest neighbor); shard embeddings.

9. Numbers to Memorize

100,000 candidates → top 100

weights 40/25/15/10/10 + RoleFit + TrapPenalty

all-MiniLM-L6-v2, 384-dim

NDCG@10 0.50, NDCG@50 0.30, MAP 0.15, P@10 0.05

~36s to rank 100K (budget 5 min / 16 GB / CPU / no net)

0 honeypots in top 100 (DQ > 10%)

15 passing tests

10. Run Commands

```
cd C:\Users\Lenovo\Desktop\TalentAI

# Produce + validate the submission
python ranking-engine\run_ranking.py --no-embeddings
python submission-generator\generate.py --team team_talentai

# Tests
python -m pytest

# App - Terminal 1 (backend, port 8000)
cd backend
python -m uvicorn app.main:app --reload --port 8000

# App - Terminal 2 (frontend, port 5173)
cd frontend
npm run dev          # open http://localhost:5173
```

TalentAI · Redrob India Runs Data & AI Challenge · MiniLM embeddings + FAISS + 5-signal weighted ranking + trap detection + grounded explanations